

Summary of Changes and Response to Reviewers

Manuscript ID: COMSI-2026-04-0112

Title: “Silent Decisions Are Type Errors: Enforcing AI Accountability via Linear Resource Types”

Author: Daniel Lau Pereira Soares

Dear Editor-in-Chief and Reviewers,

Thank you for the careful and constructive assessment of this manuscript. The revision preserves the original contribution while substantially narrowing claims that exceeded the evidence, extending the empirical evaluation, clarifying the trusted boundary and polyglot threat model, and improving the paper’s organization for the broad readership of *Computer*. The manuscript now distinguishes precisely between what the BTV type boundary establishes, what the reference implementation demonstrates, and what remains an architectural or empirical limitation.

Summary of the principal changes

1. The central type law is stated consistently as $(E \otimes C_{\text{signed}}) \multimap V$: evidence and an authority-signed compliance token are consumed to construct a verdict.
2. The Constitutional Enclosure Theorem is limited to the authorized BTV public API in Safe Rust. It does not claim that an entire polyglot system cannot bypass BTV.
3. Rust ownership is described as affine rather than strictly linear. Encapsulation, ownership transfer, restricted visibility, and lint policy reinforce the BTV boundary but do not make Rust globally linear.
4. The legal discussion now accurately describes [LGPL Article 20](#) as a right to request review and receive clear information about criteria and procedures. Human oversight is associated principally with [EU AI Act Article 14](#), and explanation rights with Article 86.
5. HMAC-based records are described as authenticated and tamper-evident under the configured key-management model, not as providing independent third-party non-repudiation.
6. The empirical section now reports concurrent-load measurements, two explicitly scoped status-quo baselines, failure behavior, durable persistence, and estimated percentiles, while stating the remaining hardware and duration limitations.
7. A polyglot/PyO3 threat-model sketch is included in the main text. It names the downstream enforcement assumption and the limits of the Rust boundary.
8. Figures were redrawn to improve readability, correct the legal citation and type-law direction, and align the benchmark visualization with the revised measured data. A new figure illustrates the polyglot/FFI trust boundary (Section 4.7), previously discussed only in prose.
9. Definitions and explanations were expanded so that governance, compliance, and systems readers can follow the argument without prior familiarity with Rust, affine ownership, FFI, or cryptographic primitives.

Response to the Editor-in-Chief

E1. Quantitative crossover claim and inaccessible supporting manuscript

Response. In response to the Editor’s concern regarding the quantitative crossover claim and its supporting unpublished companion reference, the revision removes the universal N^* headline and does not rely on an inaccessible manuscript for a load-bearing quantitative conclusion. The revised Section 5 presents the economic observation qualitatively and in both directions: the durable BTV pipeline is slower than the status quo against which it is compared ($11.1\times$ against full-context logging at 4 KiB, $155.5\times$ against digest-only logging), and the paper states plainly what that cost buys—a decision record

that provably exists. Every quantitative statement in the section traces to committed, timestamped measurement data, and the manuscript is self-contained on this point.

E2. Variance under load, baselines, and reproducibility

Response. Section 5 was substantially expanded. It now reports five-trial concurrent-load measurements at one, two, and four threads; trial-to-trial coefficient of variation; P^2 estimated p50 and p99; two status-quo baselines; and in-memory versus durable SQLite persistence. The failure experiment shows that the fire-and-forget baselines can report success while every record is lost, whereas the durable BTV path returns a verdict only after persistence succeeds.

The revised text also states the limits directly. The internally consistent current collection was produced on one cloud-hosted x86-64 virtual machine. The 90-second-per-configuration run on dedicated hardware and a second platform using the current code remain open. ARM64/QEMU is used only for compile-and-test compatibility and is not presented as ARM performance evidence. Percentiles are identified as online P^2 estimates, not exact order statistics, and no claim of linear scaling, universal speed superiority, or production-ready storage performance is made.

The construction measurements are also disambiguated. The full public path includes authority-signed token issuance and verification. The smaller prebuilt-token binding row creates tokens outside the timed region and measures only their binding into a verdict; it is not a second estimate of the end-to-end path.

E3. Polyglot and FFI threat model

Response. The main text now includes the PyO3 boundary and its explicit assumptions. Python supplies raw context to Rust; token construction, authority validation, verdict construction, and persistence occur in the protected component. A compromised orchestrator can still bypass the gateway unless the downstream decision effector accepts only a valid sealed BTV record. Enforcing that routing rule is an infrastructure obligation outside the Rust type theorem. This distinction is now part of the theorem statement, proof assumptions, discussion, and conclusion rather than being deferred to future work.

Response to Reviewer 1

We thank Reviewer 1 for recognizing the originality and relevance of the proposal and for identifying that the theoretical claims were stronger than the supporting evidence, the evaluation was prototype-level, the conclusions required further validation, and the presentation needed to be more accessible to *Computer's* broad audience.

The theoretical result is now bounded to an external Safe Rust caller using the authorized BTV API. The proof no longer treats affine Rust as strictly linear or claims system-wide impossibility of bypass. The abstract, introduction, theorem, discussion, and conclusion use the same scoped claim. The evaluation now includes contention, variance, persistence, failure behavior, and baseline comparisons, while retaining explicit limitations about duration, storage, and hardware breadth. We also reorganized and expanded the explanatory material, defined specialized terms on first use, corrected the legal characterization, and separated type-level guarantees from operational assumptions. These changes are intended to make both the contribution and its limits clear to governance, legal, and systems readers.

Response to Reviewer 2

R2.1. Linear logic versus Rust's ownership model

Response. The manuscript now states that Rust ownership is affine: a value may be used zero or one times. `#[must_use]` and `#![deny(unused_must_use)]` are guardrails at the crate boundary, not

proof that Rust is globally strictly linear. Explicit dropping, `mem::forget`, `panic`, `abort`, and `unsafe` or foreign-code behavior are addressed as limits.

R2.2. Formal assumptions and proof scope

Response. The formal section now defines the protected set as unevidenced BTV verdicts obtainable through the public API and names the assumptions: Safe Rust at the component boundary, private fields, restricted token consumption, authority-validated compliance tokens, and routing through the authorized constructor. The result is an API-enclosure argument supported by compile-fail and runtime tests; it is not presented as a mechanized proof of a complete distributed system.

R2.3. Trusted computing base, persistence, and external storage

Response. The trusted computing base is explicit: compiler and standard library, cryptographic dependencies, key management, and the configured `LogSink`. The paper distinguishes in-memory construction from fail-secure durable issuance and explains that durability is a property of the selected storage backend. The SQLite reference sink is append-only and rejects conflicting replays, but the paper does not generalize its latency or availability to production storage systems.

R2.4. Unsafe code, distributed execution, and adversarial counterexamples

Response. The core crate forbids unsafe code, while unsafe transitive dependencies remain within the trusted computing base and require supply-chain review. Counterexamples now cover token discarding, forged compliance metadata, sink failure, replay conflict, interpreter compromise, and bypass of the FFI gateway. Distributed orchestration remains outside the type theorem unless downstream enforcement makes the gateway mandatory.

R2.5. Evaluation breadth and sensitivity

Response. The revised evaluation varies payload size, thread count, persistence posture, and baseline semantics. It reports variation and tail behavior, corrects a prior durable-mode benchmark defect, and states that container-overlay `fsync` measurements are a lower bound for bare-metal storage cost. The remaining long-run and second-current-platform work is identified rather than claimed as complete.

R2.6. Strength of security and governance claims

Response. Claims of universal silent-decision elimination were replaced with the narrower prevention of unevidenced BTV verdict construction within the defined boundary. HMAC is described as providing integrity and authenticity under the configured key-management model. Independent non-repudiation would require asymmetric signatures, signer identity, key policy, and third-party verification, which the prototype does not provide. The CAL discussion is retained only as a conceptual design observation.

R2.7. Related work

Response. The related-work section now engages the four requested foundations: Ahmed, Dreyer, and Rossberg; DeLine and Fähndrich’s Vault work; Pierce; and Walker. Girard and Wadler remain as additional background. The Vault reference uses the [ACM SIGPLAN Notices record](#), DOI 10.1145/381694.378811.

Final scope statement

The revised manuscript claims that, within a Safe Rust component that preserves BTV encapsulation and routes consequential decision emission through the authorized BTV interface, an external caller cannot construct a BTV **Verdict** without the required evidence and compliance tokens. It does not claim that this local theorem proves complete governance, semantic truth of the evidence, universal system routing, third-party non-repudiation, or production-scale performance.

We appreciate the Editor's and Reviewers' guidance. Their comments resulted in a more precise, transparent, and useful manuscript.

Sincerely,

Daniel Lau Pereira Soares